

考试科目名称 编译原理; A 卷

考试方式: 闭卷 考试日期: 2021 年 01 月 17 日 教师: 魏恒峰

院系 (专业): 软件学院 (软件工程) 年级: 2018 级

姓名: _____ 学号: _____ 成绩: _____

题号	一	二	三	四	五	六	总分			
分数										

注意事项:

- 诚信考试, 不得作弊。
- 若对题意有疑问, 请及时提出。
- 题目不是按照难度排列的, 请注意合理分配时间。
- 为了避免连锁错误造成的影响, 请尽量给出关键步骤。
- 题目中的分值仅表明各小题之间的相对权重, 不代表实际分数。
- 提示: 试题后的提示仅作提示之用, 不排除其它解法。
- 字迹要尽可能工整, 解题过程要尽可能清晰。

题目 1 (正则表达式与自动机 (10 = 3 : 4 : 3))

考虑正则表达式 $r = \epsilon|(a^*b)$ 。

- (1) 请使用 Thompson 构造法构造等价的 NFA;
- (2) 请使用子集构造法构造等价的 DFA (注: 请标明状态之间的对应关系);
- (3) 请将上一步构造的 DFA 最小化。

题目 2 (LL 语法分析 ($10 = 2 : 4 : 3 : 1$))

考虑文法 G :

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow Aaa|a \quad (2)$$

$$B \rightarrow Bbb|b \quad (3)$$

- (1) 请消除文法 G 中的左递归 (记新文法为 G');
- (2) 请为文法 G' 计算必要的 FIRST 集合与 FOLLOW 集合;
- (3) 请为文法 G' 构造预测分析表;
- (4) 请问文法 G' 是否是 $LL(1)$ 文法, 并说明理由。

题目 3 (*LR* 语法分析 (10 = 2 : 3 : 2 : 3 修改为 10 = 3 : 4 : 3))

考虑文法 G (已作增广处理):

$$S' \rightarrow S \quad (0)$$

$$S \rightarrow V = E \quad (1)$$

$$S \rightarrow E \quad (2)$$

$$E \rightarrow V \quad (3)$$

$$V \rightarrow x \quad (4)$$

$$V \rightarrow *E \quad (5)$$

(1) 请为文法 G 计算必要的 FIRST 集合与 FOLLOW 集合;

(2) 请为文法 G 构造 $LR(1)$ 自动机;

注意: 为了尽量统一状态编号, 便于批改, 当计算 CLOSURE 时, 请按照文法编号大小顺序加入新项。当计算 GOTO(I, X) 时, 请按照 I 中项的出现顺序依次考虑可能的转移符号 X 。

要求: 请说明归约的设置条件。

(3) 请为文法 G 设计 $LR(1)$ 分析表; 该文法是 $LR(1)$ 文法吗? 请说明理由。如果该文法是 $LR(1)$ 文法, 请给出识别输入串 $*x = **x$ 时自动机所经历的状态 (编号)。

(4) [删除] 为该文法设计 $LALR(1)$ 分析表; 该文法是 $LALR(1)$ 文法吗? 请说明理由。

题目 4 (语法制导定义与翻译 (10 = 6 : 4))

考虑如下文法 G ,

$$\begin{aligned} S &\rightarrow (L) \mid a \\ L &\rightarrow L, S \mid S \end{aligned}$$

请设计语法制导的**翻译方案**, 完成下列任务。你需要自行定义合适的属性。

- (1) 计算每个 a 的嵌套深度。例如, 在句子 $(a, (a, a))$ 中, 每个 a 的嵌套深度分别为 1, 2, 2。
- (2) 计算每个 a 的位置。例如, 在句子 $(a, (a, (a, a), (a)))$ 中, 每个 a 的位置分别为 2, 5, 8, 10, 14。

题目 5 (中间代码生成 ($10 = 3 : 3 : 4$ 修改为 $10 = 4 : 1 : 5$))

考虑循环语句

do S **while** B

- (1) 请基于控制流语句与布尔表达式语法制导定义 (见图??与图??) 为 **do...while** 语句添加语义规则;
- (2) 请基于布尔表达式与控制流语句回填翻译方案 (见图??与图??) 为 **do...while** 语句设计回填方案。
- (3) 请为以下代码片段生成中间代码 (注: 请自行选择是否使用回填方案)

```

do
    if  $i == 2021$ 
        print "Happy New Year"
     $i = i + 1$ 
while ( $i > 2000 \ \&\& \ i < 3000$ )

```

(考试后修正: $i = i + 1$ 放在 if 外; 不影响解题)

要求: 请使用图示 (如注释语法树等) 展示产生式与相应规则的使用情况。

题目 6 (语法分析算法设计 (附加题: 5 分))

如果某上下文无关文法的每个产生式都是下列形式之一, 则该文法是 CNF (Chomsky Normal Form) 文法:

- $S \rightarrow \epsilon$, 其中 S 必须是起始符号;
- $A \rightarrow BC$, 其中 B, C 均为非终结符;
- $A \rightarrow a$, 其中 a 为终结符。

请使用动态规划思想设计语法分析算法, 判定输入串 w 是否符合某 CNF 文法 G 。

(注: 你设计的算法不要求从左到右仅扫描一遍输入串; 可以假定输入串已经全部读入。)

(提示: 考虑输入串的任一子串以及任一非终结符。)

图??与 $w = \text{she eats a fish with a fork}$ 作为例子供参考。

$S \rightarrow NP VP$
 $VP \rightarrow VP PP$
 $VP \rightarrow V NP$
 $VP \rightarrow \text{eats}$
 $PP \rightarrow P NP$
 $NP \rightarrow \text{Det } N$
 $NP \rightarrow \text{she}$
 $V \rightarrow \text{eats}$
 $P \rightarrow \text{with}$
 $N \rightarrow \text{fish}$
 $N \rightarrow \text{fork}$
 $\text{Det} \rightarrow \text{a}$

图 1: CNF 文法示例 G

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' S.next)$ $\quad \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

图 2: 控制流语句的语法制导定义

产生式	语义规则
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\quad \parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$ $\quad \parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$

图 3: 布尔表达式的语法制导定义

1) $B \rightarrow B_1 \parallel M B_2$	{ $backpatch(B_1.falselist, M.instr);$ $B.truelist = merge(B_1.truelist, B_2.truelist);$ $B.falselist = B_2.falselist;$ }
2) $B \rightarrow B_1 \&\& M B_2$	{ $backpatch(B_1.truelist, M.instr);$ $B.truelist = B_2.truelist;$ $B.falselist = merge(B_1.falselist, B_2.falselist);$ }
3) $B \rightarrow ! B_1$	{ $B.truelist = B_1.falselist;$ $B.falselist = B_1.truelist;$ }
4) $B \rightarrow (B_1)$	{ $B.truelist = B_1.truelist;$ $B.falselist = B_1.falselist;$ }
5) $B \rightarrow E_1 \text{ rel } E_2$	{ $B.truelist = makelist(nextinstr);$ $B.falselist = makelist(nextinstr + 1);$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -');$ $gen('goto -');$ }
6) $B \rightarrow \text{true}$	{ $B.truelist = makelist(nextinstr);$ $gen('goto -');$ }
7) $B \rightarrow \text{false}$	{ $B.falselist = makelist(nextinstr);$ $gen('goto -');$ }
8) $M \rightarrow \epsilon$	{ $M.instr = nextinstr;$ }

图 4: 布尔表达式的回填翻译方案

1) $S \rightarrow \text{if}(B) M S_1$	{ $backpatch(B.truelist, M.instr);$ $S.nextlist = merge(B.falselist, S_1.nextlist);$ }
2) $S \rightarrow \text{if}(B) M_1 S_1 N \text{ else } M_2 S_2$	{ $backpatch(B.truelist, M_1.instr);$ $backpatch(B.falselist, M_2.instr);$ $temp = merge(S_1.nextlist, N.nextlist);$ $S.nextlist = merge(temp, S_2.nextlist);$ }
3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$	{ $backpatch(S_1.nextlist, M_1.instr);$ $backpatch(B.truelist, M_2.instr);$ $S.nextlist = B.falselist;$ $gen('goto' M_1.instr);$ }
4) $S \rightarrow \{ L \}$	{ $S.nextlist = L.nextlist;$ }
5) $S \rightarrow A ;$	{ $S.nextlist = \text{null};$ }
6) $M \rightarrow \epsilon$	{ $M.instr = nextinstr;$ }
7) $N \rightarrow \epsilon$	{ $N.nextlist = makelist(nextinstr);$ $gen('goto -');$ }
8) $L \rightarrow L_1 M S$	{ $backpatch(L_1.nextlist, M.instr);$ $L.nextlist = S.nextlist;$ }
9) $L \rightarrow S$	{ $L.nextlist = S.nextlist;$ }

图 5: 控制流语句的回填翻译方案